Pipeline Analyzer

Project Review Book

Architecture, implementation, attack models, results, and panel preparation

Prepared from the repository source and its documented sample behavior

October 2026

How to use this book

This is a focused review guide intended to be read in about 40–55 minutes. Chapters 1–5 explain what the software does and how it is implemented. Chapter 6 explains the score and the numbers that can safely be reported. Chapter 7 is a demo script; Chapters 8–9 are a glossary and panel practice. Source citations point to project files or primary incident reports. The simulator is an educational scenario model: its outcomes describe encoded rules and a chosen graph, not real attack probability or organizational risk.

Contents

1 Project in one view **1**
- 1.1 Purpose . . . 1
- 1.2 The architecture . . . 1
- 1.3 Repository map . . . 2

2 Graph model and data flow **3**
- 2.1 Nodes and edges . . . 3
- 2.2 Storage and HTTP contract . . . 3
- 2.3 Why the API is the seam . . . 3

3 Attack simulation implementation **4**
- 3.1 Inputs and validation . . . 4
- 3.2 Traversal in plain language . . . 4
- 3.3 Four outcomes . . . 4
- 3.4 Trace fields . . . 5

4 The three modeled attacks **6**
- 4.1 ChainDrop npm worm . . . 6
- 4.2 Build Tamper (SolarWinds-style) . . . 6
- 4.3 Maintainer Takeover (xz-style) . . . 7
- 4.4 Cross-scenario control coverage . . . 7

5 Quantified results and score interpretation **8**
- 5.1 Formula . . . 8
- 5.2 Worked golden-fixture result . . . 8
- 5.3 What can and cannot be claimed . . . 8

6 Demo plan and implementation rationale **10**
- 6.1 A 7–10 minute walkthrough . . . 10
- 6.2 Why these implementation choices . . . 10
- 6.3 Known modeling limits . . . 11

7 Glossary for memorization **12**

8 Panel questions and concise answers **13**

Sources and further reading **15**

Chapter 1

Project in one view

1.1 Purpose

Pipeline Analyzer is a browser-based, interactive graph model of a software delivery environment. The user places components, connects them with directed relationships, edits security properties, selects one of three attack scenarios, and reads a trace showing which modeled steps compromise, slow, block, or detect an attack. This supports explaining trust relationships and comparing controls in a repeatable demonstration.

It is not a scanner, exploit framework, threat-intelligence feed, probabilistic risk engine, or proof that a control is effective in a real deployment. A result is conditional on graph completeness, property values, scenario rules, and implementation assumptions.

1.2 The architecture

Frontend	React 19 and TypeScript interface; Zustand holds projects, graph, editor state, and simulation playback state. Vite builds and serves the frontend.
Graph display	HTML canvas rendering, a D3 force simulation in a Web Worker, graph search, selection, and relationship validation. The canvas is for layout and interaction; attack logic is in Rust.
API	Rust 2024 with Axum/Tokio. JSON HTTP routes serve metadata, graph CRUD, relationship validation, and attack simulation on 127.0.0.1:3000.
Persistence	JSON file store (db.json). A Tokio RwLock guards in-memory state; updates serialize and write a temporary file before rename.
Environment	Nix flake supplies Rust tooling, Node 24, pnpm, and texliveSmall.

The browser initializes by fetching node categories, relationship names, and scenario metadata, then loads projects and the selected graph. Metadata comes from the backend schema so the frontend can render editable fields without duplicating the domain definitions. An attack request sends project ID, start node ID, and scenario. The backend snapshots the selected project's graph, simulates it, and returns a trace DTO; the frontend animates that trace and shows the controls and score.

1.3 Repository map

`backend/src/schema.rs`	Category properties, defaults, and allowed relationship pairs.
`backend/src/attack/mod.rs`	Scenario metadata, route filtering, rule evaluation, traversal, trace construction, and scoring.
`backend/src/lib.rs, main.rs`	HTTP handlers, state, routes, startup, and project CRUD.
`backend/src/types.rs`	Serialized graph, API, trace, outcome, and error data types.
`backend/tests`	Integration/API checks and the ChainDrop golden JSON result.
`frontend/src/api/client.ts`	HTTP client plus translation between backend DTOs and frontend types.
`frontend/src/store/graphStore.ts`	Application and graph state/actions, including simulation requests and playback.
`frontend/src/components`	User interface, details panel, graph canvas.
`frontend/src/engine`	Canvas graph engine, renderers, search, and physics worker.

Chapter 2

Graph model and data flow

2.1 Nodes and edges

A graph is a directed multigraph-like structure: nodes represent people, source control, dependencies, build environments, CI pipelines, credentials, artifact storage, distribution, production runtime, and monitoring; edges represent typed relations such as `consumed_by`, `triggers`, `authenticates`, `publishes`, and `deploys`. Edges have source and target node IDs plus a relationship ID. The schema currently exposes 10 categories and 22 relationship IDs (also asserted by the API tests).

Properties are JSON-compatible string, number, or boolean values. Category metadata defines each property's editor type, options, and default. The backend is the source of truth for property definitions and valid source-category/target-category relationship pairs. The frontend asks the API to validate candidate connections and lets the user choose among allowed relation labels.

2.2 Storage and HTTP contract

Projects, nodes, edges, and project ownership maps are held in the JSON database. The store uses a reader/writer lock. For a write, the lock is held, the in-memory database is changed, JSON is serialized, written to a sibling temporary path, and renamed over the data file. This reduces the chance of a partially written main file if interrupted. The implementation notes that an I/O failure after in-memory mutation does not roll back the in-memory change.

Important routes include `GET /node-types`, `GET /relationships`, `GET /scenarios`, `GET /relationships/validate`, project CRUD under `/projects`, graph retrieval at `/projects/{id}/graph`, and `POST /projects/{id}/attack`. API errors map common missing resources to 404, invalid inputs to 400, and duplicate edges to 409.

2.3 Why the API is the seam

The React app owns presentation, editing, and playback. The Rust backend owns canonical schema, persistence, validation, and simulation. That division keeps scenario results deterministic for a given request graph and makes the attack rules independently inspectable. The frontend DTO adapter handles the Rust serializer's PascalCase response fields and maps them into UI-oriented types.

Chapter 3

Attack simulation implementation

3.1 Inputs and validation

The simulator takes a graph, attack ID, start node ID, and scenario string. Scenario parsing is case-insensitive and ignores spaces, hyphens, and underscores. It rejects unknown scenarios, missing start nodes, and start categories not permitted by that scenario. ChainDrop accepts a dependency or person; Build Tamper requires a build environment; Maintainer Takeover requires a person.

3.2 Traversal in plain language

The core routine uses a FIFO queue (breadth-first traversal). The start node is put into the compromised set, visited set, and queue. For each queued node it considers outgoing edges, skips edges already traversed, confirms that the scenario follows that edge/target category, evaluates the hop, records a trace step, and applies the outcome. A set prevents re-queuing a node, while a separate edge set prevents evaluating an edge more than once. Blocked branches do not enqueue their target. Slowed and detected targets do enqueue and remain part of the compromised count.

Scenario filtering and hop evaluation are separate: `follows` determines whether an edge is in scope; scenario evaluators determine outcome/reason/controls for in-scope hops. Most uneventful hops fall through to shared base rules. This makes trace explanations and scenario-specific exceptions explicit.

ChainDrop adds reverse credential harvesting. When a compromised build environment or CI node has an incoming `authenticates` edge from a credential, the simulator traces from the job back to that credential. It then checks a compromised credential's token scope when evaluating republishing. Trusted OIDC publishing or a publish plus 2FA-bypass credential permits propagation; granular scope slows it; read-only or absent publish authority blocks it. The result exposes IDs of harvested secrets and a worm-generation count.

3.3 Four outcomes

Outcome	Effect on target	Can traversal continue from target?
Compromised	Add target to compromised set	Yes
Slowed	Add target to compromised set; mark friction in the trace	Yes
Blocked	Add target to blocked set	No
Detected	Add target to compromised set and record first detection index	Yes

Detection is not prevention in this model. Also, only the first detection index is retained; a later detection may still appear as a trace step but does not replace it.

3.4 Trace fields

Each step carries edge ID, source and target IDs, relationship, outcome, human-readable reason, controls triggered, optional phase, and optional reverse marker. This is a notable design choice: the system returns an explanation alongside each modeled outcome, rather than only a final score. The UI can animate the step sequence and focus the relevant graph nodes.

Chapter 4

The three modeled attacks

These are intentionally compact teaching scenarios inspired by incidents. Their names and resemblance do not mean the software reproduces the full incident.

4.1 ChainDrop npm worm

Question it explores: how can a malicious dependency execute in a developer/CI environment, expose a publishing credential, and spread into dependent packages?

Modeled sequence: malicious dependency – installation relationship – developer or build/CI – credential harvesting – credential use – downstream republishing – artifact/distribution/runtime. The start may be a dependency or person. Lifecycle scripts set to Disabled block the install execution path. A non-None release cooldown together with lockfile pinning blocks resolution in the rule; either control alone slows it. The vulnerability-scanning property does not stop a just-published package without a known CVE, by design. Trusted publishing can create a validly attested poisoned release, so signature verification is not treated as proof of benign contents. Integrity checking may match the already-poisoned published hash. Credential lifetime and scope affect harvesting/propagation as described above.

Real-world analogue: Shai-Hulud campaigns used npm lifecycle scripts, credential theft, and self-propagation. GitHub describes malicious post-install scripts used to exfiltrate secrets and replicate across the JavaScript package ecosystem. The 2025 campaigns are analogues for the chain of trust; they are not the same attack as this simulator's named ChainDrop scenario. [GitHub campaign analysis and mitigations] [Microsoft security analysis]

4.2 Build Tamper (SolarWinds-style)

Question it explores: what if the build system, not the source repository or delivery channel, is compromised?

Modeled sequence: compromised build environment – artifact registry – distribution/runtime, with optional monitoring. Setting `reproducibleBuilds` on the build environment blocks publication because the independent rebuild is modeled to produce a different hash. Without that, a validly signed poisoned build is accepted even with signature verification enabled: signature establishes publisher/provenance in this model, not that content is benign. Delivery integrity checks match the poisoned release. Zero-Tolerance alerting on the monitoring node detects the modeled anomalous build behavior, but detection does not stop continued traversal.

Real-world analogue: CISA reported that malicious code was inserted into SolarWinds Orion updates distributed to customers through the trusted software update process. The actual operation included extensive, sophisticated follow-on activity and is much more complex than this graph path. [CISA SolarWinds alert] [CISA supply-chain defense guidance]

4.3 Maintainer Takeover (xz-style)

Question it explores: can a trusted maintainer or release process publish an artifact that differs from what reviewers saw in source history?

Modeled sequence: trusted person – repository – release CI – artifact store – distribution/runtime. With 2+ reviewers, the repository hop is Slowed but still traversable; the rule treats a trusted maintainer's compromise as able to evade the intended review. Branch protection is recorded as bypassed for that trusted maintainer in this scenario. If the workflow uses `Built from tag`, the hidden payload in an uploaded tarball is blocked at publication. A valid signature and integrity match do not reveal that the released object differs from reviewed source because they attest to the accepted object/process, not semantic safety.

Real-world analogue: Andres Freund's March 2024 disclosure described malicious code in XZ Utils/liblzma releases and its potential to compromise SSH authentication in affected Linux environments. His report is the primary technical disclosure. The simulator abstracts the trust-building and release/archive mismatch issues; it does not claim XZ followed this exact workflow. [Freund's Openwall disclosure] [OpenSSF/OpenJS maintainer alert]

4.4 Cross-scenario control coverage

Control in the sample	ChainDrop	Build Tamper	Maintainer Takeover
Cooldown plus lockfile pin	Blocks fresh resolution	–	–
Signature verification	No effect on trusted poisoned release	No effect on validly signed build	No effect on legitimate publisher
Integrity check	Matches poisoned published object	Matches poisoned release	Matches poisoned release
Reproducible builds	–	Blocks registry publication	–
Release built from reviewed tag	–	–	Blocks uploaded archive route
2+ reviewers	–	–	Slows, does not block
Zero-Tolerance monitoring	Detects if route reaches monitored node	Detects modeled anomaly	No effect in the trusted-access rule

The intended lesson is layered controls: provenance, integrity, review, reproducibility, credential scope, and monitoring answer different questions. No single property protects all modeled paths.

Chapter 5

Quantified results and score interpretation

5.1 Formula

Let C be the number of compromised nodes, N the total number of nodes in the selected graph, and D equal 1 if any detection occurred and 0 otherwise. The implementation computes:

$$S_{base} = 10 \times \frac{C}{\max(N, 1)}, \qquad S_{final} = S_{base} \times \begin{cases} 0.7, & D = 1, \\ 1, & D = 0. \end{cases}$$

Labels use the final score: Critical for $S \geq 7.5$; High for $5 \leq S < 7.5$; Medium for $2.5 \leq S < 5$; Low for $S < 2.5$. The start node is counted. Slowed and Detected targets count as compromised; Blocked targets do not. All graph nodes are in the denominator, including scenario-irrelevant and unreachable nodes.

5.2 Worked golden-fixture result

The checked-in ChainDrop fixture contains 12 graph nodes, 18 trace steps, 10 compromised node IDs, one blocked node ID, one detection (step index 16 in the zero-based serialized trace), two worm generations, and one exposed secret ID. So:

$$10 \times \frac{10}{12} = 8.3333\ldots, \quad 8.3333\ldots \times 0.7 = 5.8333\ldots$$

The label is High. This is a fixture result for its graph and property values, not a measured attack rate. It is useful as an exact example of the algorithm and an API regression reference.

5.3 What can and cannot be claimed

- Safe quantified claims: the model has 3 scenarios, 10 node categories, and 22 relationship IDs; the golden ChainDrop fixture has 18 evaluated hops and score 5.833 (High); the score formula uses the compromised fraction and a fixed 30% discount when detection occurs.
- Unsafe claims: “the project reduces risk by 30%,” “the attack has a 58.3% probability,” or “High means a real incident is High.” Those interpretations are not represented by the code.
- Scores depend on graph size: adding unrelated nodes lowers C/N without changing the reachable attack path. Compare before/after results on the same graph and scenario, and inspect the trace as well as the score.

- Detection discount is global and binary: one detected hop multiplies the whole base score by 0.7, regardless of when detection happens, how many nodes were detected, or whether response stopped the attack.

Chapter 6

Demo plan and implementation rationale

6.1 A 7–10 minute walkthrough

1. Load the sample graph. Briefly identify dependency, developer/build, CI, credential, registry, distribution, production, and SIEM nodes.

2. Run ChainDrop from the dependency. Read one execution hop, the reverse credential-harvest hop, and a republish hop. Set cooldown to 7 days and pin the lockfile; rerun and show the branch blocking. Explain why either alone is only a slowdown in this model.

3. Choose Build Tamper and the recommended Shared CI Runner start. Point out that signing passes for the poisoned build. Enable reproducible builds and rerun to show rejection at the registry. Then demonstrate Zero-Tolerance monitoring and clarify that detection is not containment.

4. Choose Maintainer Takeover and the recommended Package Maintainer start. Set reviewers to 2+ and show Slowed. Then set the workflow's release source to Built from tag and show the block at publication.

5. Close by comparing control coverage: the controls protect different transitions, so defense in depth is the project takeaway.

6.2 Why these implementation choices

Directed typed edges A delivery pipeline has directional dependencies and permissions. Typed edges distinguish "uses" from "publishes" and let rules match meaningful transitions.

Schema from backend Node properties and allowed edge pairs remain canonical in one place; the UI learns the editor fields through API metadata.

Queue-based graph traversal Breadth-first traversal handles branching paths, while visited nodes and traversed edges prevent cycles from creating infinite work or duplicate trace actions.

Outcome plus reason A hop-by-hop trace is more explainable in a demo than a bare terminal score and makes assumptions auditable.

Scenario-specific evaluators Shared base logic captures common control behavior, while explicit overrides model where a control is ineffective against a particular trust abuse.

Atomic file replacement Writing a temp file and renaming reduces partial-file corruption risk in the simple local persistence design.

Canvas plus worker Graph drawing and force layout stay responsive for interactive editing; the layout worker does not decide attack semantics.

6.3 Known modeling limits

This implementation is a deterministic rules engine. It does not estimate probability, exploitability, dwell time, monetary impact, asset criticality, attacker adaptation, control reliability, incident-response latency, or correlated failures. Some rules intentionally simplify nuanced controls (for example, a single reproducibility boolean stands in for an independent reproducible-build process). Branch protection and reviewer requirements are abstractions. Monitoring detects but does not automatically contain. File JSON persistence is appropriate for a small demo, not a multi-user production database. The broad CORS policy and local bind reflect a development app and should be reconsidered before exposure beyond localhost.

Chapter 7

Glossary for memorization

Artifact A built or packaged output that can be stored, signed, delivered, and installed.

Attestation/provenance Claims or records describing where/how an artifact was produced. Provenance supports origin verification; it does not prove the source or builder is benign.

Attack surface The set of possible entry points and reachable transitions represented in a system.

Directed graph A set of vertices (nodes) and oriented edges; direction indicates modeled movement or dependency.

Edge A typed directed relationship from one node to another.

Lifecycle script Package-manager hook that may run during install, such as npm preinstall/postinstall.

Lockfile pinning Recording exact dependency resolutions so builds do not silently float to a new version.

OIDC trusted publishing Short-lived identity-based publication authorization that avoids a long-lived stored token; it can still publish malicious content if the authorized workflow is compromised.

Reproducible build A build process where independent builds from the same inputs produce matching outputs; mismatch can indicate an untrusted or non-deterministic build path.

SBOM Software bill of materials: a structured inventory of software components in a product.

Signature A cryptographic check binding an artifact to a signing identity/key and detecting changes after signing; it is not a semantic safety verdict.

Supply-chain attack An attack that abuses a supplier, dependency, development, build, or distribution path to affect downstream users.

Trusted maintainer In this model, a person whose existing authority makes a malicious change appear to pass ordinary trust and release mechanisms.

Worm generation The simulator's count of successful or slowed ChainDrop republish transitions; it is not a biological-style generation time or observed package count.

Chapter 8

Panel questions and concise answers

1. **What problem does this project solve?** It gives a visual, editable way to reason about how compromise can move through a software delivery graph and how selected controls change the modeled path.
2. **What are the three scenarios?** ChainDrop npm Worm models install-hook execution, credential exposure, and republishing; Build Tamper models a compromised build and trusted poisoned artifact; Maintainer Takeover models abuse of trusted maintainer/release access and an artifact mismatch.
3. **What does a node represent?** A component or actor in delivery, such as a person, source repository, dependency, build runner, CI pipeline, credential, registry, distributor, production runtime, or monitoring system.
4. **What does an edge represent?** A directed typed relationship that encodes a modeled dependency, action, permission, or delivery transition.
5. **State the score formula.** $S = 10(C/N)$, then multiply by 0.7 if any hop was detected; C includes the start, slowed, and detected nodes, but excludes blocked nodes.
6. **What are the thresholds?** Low below 2.5, Medium from 2.5 to below 5, High from 5 to below 7.5, Critical at 7.5 or above.
7. **What did the golden example produce?** 12 nodes; 10 compromised; 1 blocked; 18 evaluated steps; first detection at step index 16; 2 worm generations; one exposed secret; final score 5.833, High.
8. **Is that an empirical risk result?** No. It is deterministic output from one fixture graph and fixed rules. It is a model reach score, not likelihood, expected loss, or a measured risk reduction.
9. **Why does detection still count as compromised?** The simulator models detection as observation, not automatic prevention or response. The target remains reachable and traversal continues.
10. **Why doesn't signature verification block a poisoned build?** A signature can validate the authorized publisher and post-signing integrity. In the Build Tamper scenario, the compromised build is validly signed, so authenticity alone does not establish benign content.
11. **Why does the integrity check not help?** It checks that the delivered object matches the published object. If the published object is already malicious, a matching hash confirms consistency with that malicious object.

12. **What blocks Build Tamper in the sample?** Reproducible builds on the build environment; independent rebuild mismatch rejects publication at artifact storage.

13. **What blocks Maintainer Takeover?** Requiring the release to be built from the reviewed tag, which prevents the unreviewed uploaded archive route in this model. Two reviewers only slows it.

14. **What does the ChainDrop reverse edge mean?** The forward graph edge says a credential authenticates to a build/CI node. During harvesting, traversal follows that relationship backward from compromised job to credential to represent secret exposure.

15. **How is worm generation counted?** Each ChainDrop `republishes` hop with outcome Compromised or Slowed increments the counter; blocked republishes do not.

16. **Why BFS and sets?** A FIFO queue explores reachable nodes; a visited set queues each node once and a traversed-edge set records each edge at most once, handling branching and cycles predictably.

17. **Where should a new node property be added?** In `backend/src/schema.rs`; the backend serves it as metadata for the frontend editor.

18. **Where should a new scenario rule go?** In `backend/src/attack/mod.rs`: scenario enum/metadata/start categories, route filter, evaluator, phase mapping as needed, and test coverage/fixture updates.

19. **What does the frontend own?** Graph editing and visualization, API calls and DTO mapping, application state, and attack-trace playback. It does not calculate scenario outcomes.

20. **How is project state persisted?** A JSON file stores projects/nodes/edges; Tokio RwLock protects the in-memory database and persistence writes a temporary JSON file then renames it.

21. **Name one limitation.** The graph and rules omit probability, impact, timing, attacker adaptation, and control failure rates. Other acceptable answers include global detection discount, denominator sensitivity, simplified controls, or local JSON storage.

22. **What is the main security lesson?** Controls are transition-specific: signed provenance, reproducibility, reviewed-source release, dependency pinning, credential scope, and monitoring protect different parts of the pipeline and should be layered.

23. **How do you defend the incident comparisons?** They are analogies used to teach classes of trust failure. The scenario logic is a simplified abstraction and does not claim to reproduce an incident's chronology or full tradecraft.

Sources and further reading

1. Project README, introduction, and onboarding: repository files `README.md`, `INTRO.md`, and `ONBOARD.md` in the checked-out project.
2. Project implementation: `backend/src/attack/mod.rs`, `backend/src/schema.rs`, `backend/src/types.rs`, `backend/src/lib.rs`, `backend/src/main.rs`; test references in `backend/tests/api_tests.rs` and `backend/tests/fi`
3. CISA, "Active Exploitation of SolarWinds Software," 2020: https://www.cisa.gov/news-events/alerts/2020/12/13/active-exploitation-solarwinds-software.
4. CISA, "Defending Against Software Supply Chain Attacks": https://www.cisa.gov/sites/default/files/publications/defending_against_software_supply_chain_attacks_508_1.pdf.
5. Andres Freund, XZ Utils/liblzma backdoor disclosure, Openwall oss-security, 29 March 2024: https://www.openwall.com/lists/oss-security/2024/03/29/4.
6. OpenSSF/OpenJS, alert on social-engineering takeovers of open-source projects, 15 April 2024: https://openssf.org/blog/2024/04/15/open-source-security-openssf-and-openjs-foundations-iss
7. GitHub, "Strengthening supply chain security: Preparing for the next malware campaign" (Shai-Hulud reporting and mitigations): https://github.blog/security/supply-chain-security/strengthening-supply-chain-security-preparing-for-the-next-malware-campaign/.
8. Microsoft Security Blog, Shai-Hulud 2.0 analysis and guidance: https://www.microsoft.com/en-us/security/blog/2025/12/09/shai-hulud-2-0-guidance-for-detecting-investigating-and-defe